

Трансляция презентации (во время очных лекций)

<https://clck.ru/3D3Efj>



При просмотре презентации в PDF для отображения анимаций на слайдах необходимо использовать Acrobat Reader, KDE Okular, PDF-XChange, Foxit Reader, браузер Firefox. Для браузеров на движке Chrome (Edge, Яндекс, Opera, ...) необходимо использовать [PDF.js](#) с опцией «Enable active content (JavaScript) in PDFs».



Промышленное программирование

Тема 3

Системы управления версиями

Гаврилов Андрей Геннадьевич
кандидат технических наук, доцент

Кафедра Информационных технологий и вычислительных систем
МГТУ «СТАНКИН»

Обновлено: 26 июля 2026 г.

1 Git

2 Коммиты в git

3 Ветки в git

Раздел 1

Git

git

git —

это распределенная система контроля версий, разработанная Линусом Торвальдсом в 2005 году для управления исходным кодом ядра Linux.

Предназначение:

- Отслеживание изменений в файлах
- Совместная работа над проектами
- Возможность возврата к предыдущим версиям

Установка Windows (через актуальную(!) версию Windows Installer):

```
> winget install Git.Git
```

Установка Linux (через apt):

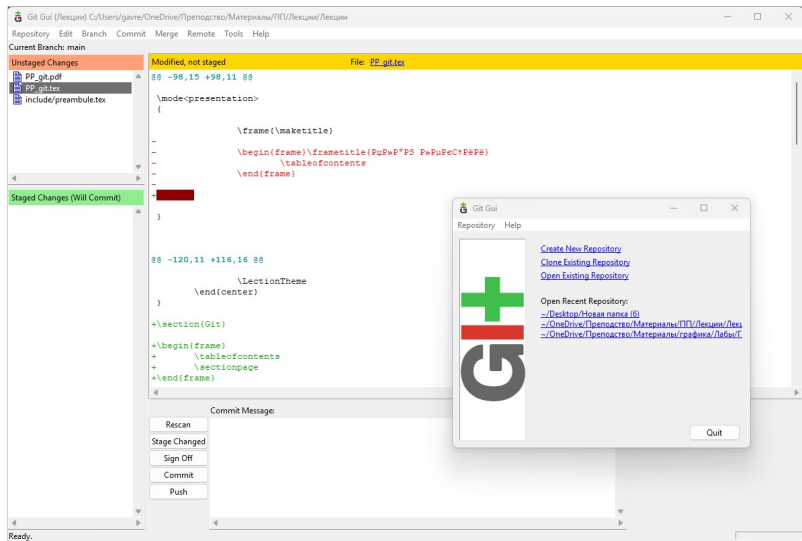
```
> sudo apt update  
> sudo apt install git
```

Установка Mac (через Homebrew):

```
> brew install git
```

Работа через графический интерфейс

Но так никто не делает

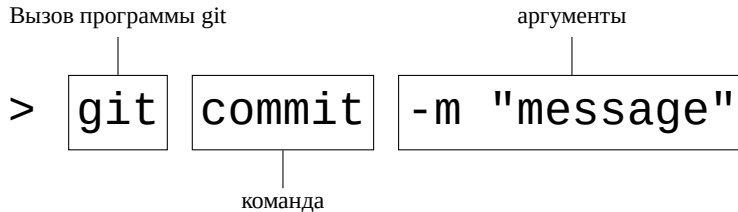


Работа через командную строку

Проверка установки

```
> git --version  
git version 2.48.1.windows.1
```

Основной синтаксис



Получить справку по любой команде git

```
> git команда --help #Полная онлайн справка  
> git команда -h #Сокращённая, в терминале
```

Настройка git

git config

позволяет задавать настройки git на уровне системы или отдельного репозитория.

Настройка имени пользователя и эл. почты на уровне системы

```
> git config --global user.name "gavreg"
> git config --global user.email "a.gavrilov@stankin.ru"

> git config --global --list
...
user.email=a.gavrilov@stankin.ru
user.name=gavreg
...
```

```
> git config --global my.var "СТАНКИН"
> git config --global --list
...
my.var=Станкин
...
```

1 Git

2 Коммиты в git

3 Ветки в git

Раздел 2

Коммиты в git

Первый коммит

1. Создадим файл ch1.txt

ch1.txt(версия 1)

Вечерело, на улице было холодно.

Папка

+ch1.txt(1)

Индекс

Репозиторий

Первый коммит

1. Создадим файл ch1.txt

ch1.txt(версия 1)

Вечерело, на улице было холодно.

2. Инициализация репозитория

```
> git init
```

git init

команда инициализации репозитория. В текущей папке создается скрытая папка .git, в которой будет размещён новый пустой репозиторий.

Папка

ch1.txt(1)

Индекс

Репозиторий

Первый коммит

3. Добавим файл ch1.txt в индекс.

```
git add —
```

команда Git, осуществляющая добавление файлов в индекс (stage).

```
> git add ch1.txt  
> git status
```

```
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
    new file:   ch1.txt
```

Папка

ch1.txt(1)

Индекс

ch1.txt(1)

Репозиторий



Первый коммит

4. Применяем (фиксируем) изменения.

Фиксация изменений (commit changes) —

процедура переноса файлов из индекса в репозиторий. Перенос осуществляется хитро — переносятся только отличия файлов индекса относительно тех, что есть в репозитории.

Коммит (commit) —

содержит описание изменений файлов текущей версии проекта относительно предыдущей. Любой коммит, кроме самого первого, основан на более раннем (родительском).

Папка

ch1.txt(1)

Индекс

ch1.txt(1)

Репозиторий

Первый коммит

4. Применяем (фиксируем) изменения.

```
git commit
```

создаёт новый коммит, перенося в репозиторий то, что хранится в индексе в индексе.

```
> git commit -m "Мой первый коммит"  
[master (root-commit) 127e c2c] Мой первый коммит  
1 file changed, 1 insertion(+)  
create mode 100644 ch1.txt
```

127e HEAD
master

Папка

ch1.txt(1)

Индекс

ch1.txt(1)

Репозиторий

ch1.txt(1)

Второй коммит

1. Изменим файл ch1.txt

ch1.txt(версия 2)

Вечерело, на улице было ~~холодно~~ жарко.

Создадим файл ch2.txt

ch2.txt(версия 1)

Однажды, в студёную зимнюю пору я из лесу вышел.

`git add ch1.txt ch2.txt` — добавить эти файлы в индекс

`git add *.txt` — добавить все txt файлы в индекс

`git add -u` — добавить все отслеживаемые (те что уже в индексе) файлы в индекс

127e HEAD
master

Папка

ch1.txt(1→2)
+ch2.txt(1)

Индекс

ch1.txt(1)

Репозиторий

ch1.txt(1)

Второй коммит

2. Обновим индекс

```
> git add ch1.txt ch2.txt
> git status
On branch master
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
    modified:   ch1.txt
    new file:   ch2.txt
```

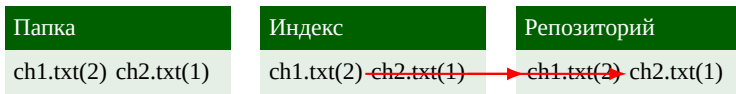
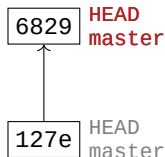
127e HEAD
master

Папка	Индекс	Репозиторий
ch1.txt(2) ch2.txt(1)	ch1.txt(2) ch2.txt(1)	ch1.txt(1)

Второй коммит

3. Создаем коммит

```
> git commit -m "Второй коммит"  
[master 68297fe] Второй коммит
```



Просмотр истории

1. Изменим файл ch1.txt

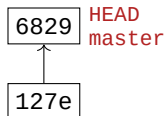
ch1.txt(версия 3)

Вечерело, на улице ~~было жарко~~ шёл дождь.

Изменим файл ch2.txt

ch2.txt(версия 2)

Однажды, в студёную зимнюю пору
Я из лесу вышел; был сильный мороз.



Папка

ch1.txt(2→3)
ch2.txt(1→2)

Индекс

ch1.txt(2) ch2.txt(1)

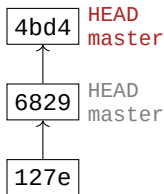
Репозиторий

ch1.txt(2) ch2.txt(1)

Просмотр истории

2. Сразу закоммитим изменения

```
> git commit -am "Коммит номер три"  
[master 4bd412d] КОММИТ номер три  
2 files changed, 3 insertions(+), 2 deletions(-)
```



Папка	Индекс	Репозиторий
ch1.txt(3) ch2.txt(2)	ch1.txt(3) ch2.txt(2)	ch1.txt(3) ch2.txt(2)

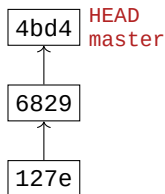
Просмотр истории

3. Посмотрим лог text

git log —

основная команда в Git для просмотра истории коммитов (фиксаций изменений) в репозитории.

```
> git log
commit 4bd412d5bfbeb8546f9e383ee6319802d7c2bee3 (HEAD -> mas
Author: gavreg <a.gavrilov@stankin.ru>
Date: Thu Mar 19 00:12:14 2026 +0300
...
```



Полный лог

Папка

ch1.txt(3) ch2.txt(2)

Индекс

ch1.txt(3) ch2.txt(2)

Репозиторий

ch1.txt(3) ch2.txt(2)

Просмотр коммита

Просмотр коммита

```
git show —
```

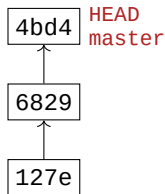
это команда Git, предназначенная для детального просмотра информации об объектах (коммитах, тегах, файлах) в удобном для чтения формате.

Самый частый вариант синтаксиса:

```
git show объект
```

Объектом может являться практически любая сущность git.

Коммит может определяться хешем, именем (или веткой), тегом.



Папка
ch1.txt(3) ch2.txt(2)

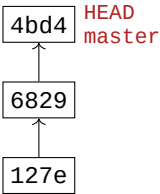
Индекс
ch1.txt(3) ch2.txt(2)

Репозиторий
ch1.txt(3) ch2.txt(2)

Просмотр коммита

Адресация коммитов:

- По хешу
`git show 127e` *Пример*
`git show 6829` *Пример*
`git show 4db4` *Пример*
- По ветке или тегу
`git show master`
- По специальному указателю
`git show HEAD == git show @`
- Выражения вида «коммит $\sim n$ », $n \in \mathbb{N}$
`git show @~2` — 2 шага до HEAD
`git show 6829~1` — 1 шаг до 6829
и.т.д.



Папка	Индекс	Репозиторий
ch1.txt(3) ch2.txt(2)	ch1.txt(3) ch2.txt(2)	ch1.txt(3) ch2.txt(2)

1 Git

2 Коммиты в git

3 Ветки в git

Раздел 3

Ветки в git

Ветки. Основные операции и команды

Ветка (branch) —

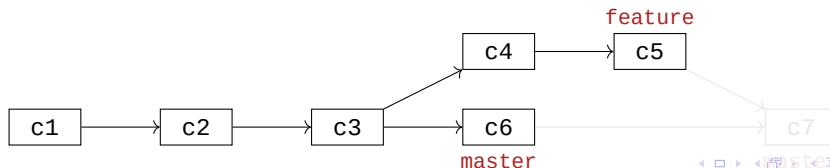
легковесный подвижный указатель на конкретный коммит в истории разработки. Она представляет собой изолированную линию разработки, позволяющую вести работу параллельно с другими линиями без влияния на них.

git branch —

команда для управления ветками в Git. Позволяет создавать, просматривать, переименовывать и удалять ветки, но не переключается между ними.

git checkout —

git checkout — многофункциональная команда для: переключения между ветками, восстановления файлов, перемещения по истории коммитов



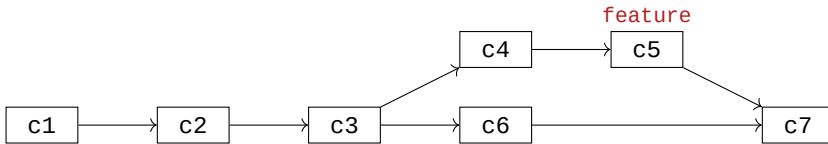
Ветки. Основные операции и команды

Слияние веток —

процесс объединения изменений из двух разных веток в одну. В системе Git это один из ключевых механизмов, позволяющий работать над новыми функциями, исправлениями или экспериментами изолированно, а затем интегрировать результат в основную линию разработки.

git merge —

используется для того, чтобы взять изменения из одной или нескольких веток и интегрировать их в текущую активную ветку.



Наименования веток

Как называть ветки:

- 1 Избегайте заглавных для единообразия.
- 2 Не используйте пробелы.
- 3 Название должно отражать суть: ~~fix~~ fix-login-bug.
- 4 Только буквы, цифры, дефисы, подчёркивания, слеш.
- 5 Не начинайте с дефиса.

Типовые префиксы:

- feature/ — новая функциональность;
- bugfix/ — исправление ошибки;
- hotfix/ — критическое исправление в production;
- release/ — подготовка релиза;
- refactor/ — рефакторинг кода;
- docs/ — обновление документации;
- test/ — добавление или изменение тестов;
- chore/ — обслуживание проекта (зависимости, конфиги);
- experiment/ — Экспериментальные изменения;
- wip/ — work in progress — незавершённая работа;

Создание веток

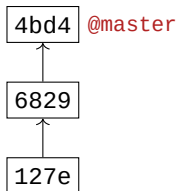
Изменим репозиторий

ch1.txt (Версия 3 → 4)

~~Вечерело~~ Наступило утро, на улице шёл дождь.

ch2.txt(версия 2)

Однажды, в студёную зимнюю пору
Я из лесу вышел; был сильный мороз.



Папка

ch1.txt(3 → 4)
ch2.txt(2)

Индекс

ch1.txt(3) ch2.txt(2)

Репозиторий

ch1.txt(4) ch2.txt(2)

Создание веток

Создадим ветку

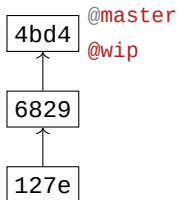
```
git checkout -b имя_ветки —
```

создаёт на текущем коммите (HEAD) новую ветку и переключает на неё HEAD.

```
git branch имя_ветки —
```

создаёт на текущем коммите новую ветку, но **не переключает** на неё HEAD.

```
PS > git checkout -b wip
```



Папка

ch1.txt(4) ch2.txt(2)

Индекс

ch1.txt(3) ch2.txt(2)

Репозиторий

ch1.txt(4) ch2.txt(2)

Создание веток

Закоммитим в ветку

```
PS > git commit -am "Перефразировал первую главу"
```

```
PS > git log
```

```
commit 8e965b9.... (HEAD -> wip)
```

```
...
```

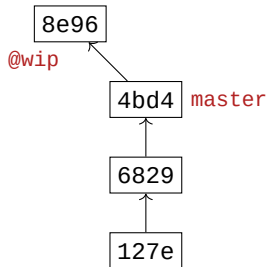
```
Перефразировал первую главу
```

```
commit 4bd412d5b.... (master)
```

```
...
```

```
Коммит номер три
```

```
...
```



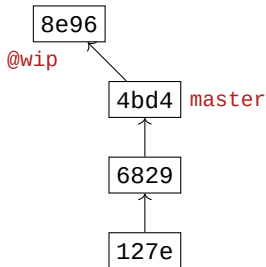
Папка	Индекс	Репозиторий
ch1.txt(4) ch2.txt(2)	ch1.txt(4) ch2.txt(2)	ch1.txt(4) ch2.txt(2)

Создание веток

Просмотр веток

```
PS > git branch
master
* wip
```

```
PS > git show-branch
[master] Коммит номер три
--
* [wip] Перефразировал первую главу
+* [master] Коммит номер три
```



Папка

ch1.txt(4) ch2.txt(2)

Индекс

ch1.txt(4) ch2.txt(2)

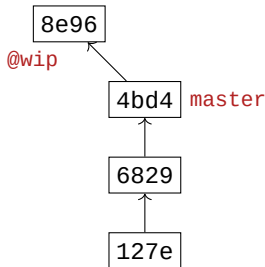
Репозиторий

ch1.txt(4) ch2.txt(2)

Переключение между ветками

1. Посмотрим файлы из других коммитов

```
PS > cat ch1.txt      #файл в директории
Наступило утро, на улице шёл дождь
PS > git show :ch1.txt #файл в индексе
Наступило утро, на улице шёл дождь
PS > git show "@:ch1.txt" #файл в HEAD
Наступило утро, на улице шёл дождь
PS > git show "@~1:ch1.txt" #файл до HEAD
Вечерело, на улице шёл дождь
PS > git show "master:ch1.txt" #файл в master
Вечерело, на улице шёл дождь
```



Папка

ch1.txt(4) ch2.txt(2)

Индекс

ch1.txt(4) ch2.txt(2)

Репозиторий

ch1.txt(4) ch2.txt(2)

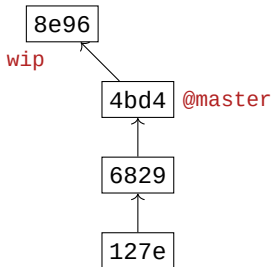
Переключение между ветками

2. Переключемся на master

```
PS > git checkout master  
Switched to branch 'master'
```

```
PS > cat ch1.txt  
Вечерело, на улице шёл дождь
```

```
PS > git show wip:ch1.txt  
Наступило утро, на улице шёл дождь
```



Папка	Индекс	Репозиторий
ch1.txt(4→3) ch2.txt(2→2)	ch1.txt(4→3) ch2.txt(2→2)	ch1.txt(4→3) ch2.txt(2→2)

Другие команды для переключения

git switch —

команда Git, появившаяся в версии 2.23 (2019 год) для переключения между ветками. Она была выделена из более универсальной команды git checkout

- Переключение на существующую ветку:
`git switch название_ветки`
- Создание и переключение на новую ветку:
`git switch -с новая_ветка`
(аналог `git checkout -b`)
- Переключение на предыдущую ветку:
`git switch -`
(возвращает на ветку, на которой вы были до этого)
- Принудительное переключение (отмена локальных изменений):
`git switch -f ветка`
(отбрасывает незакоммиченные изменения)

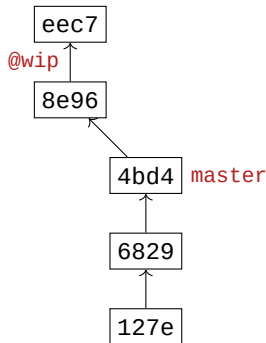
Удаление веток (DANGER!!)

1. Изменим файл и закоммитим на wip

ch2.txt(версия 2 → 3)

Однажды, в студеную зимнюю пору
Я из лесу вышел; был сильный мороз.
Гляжу, поднимается медленно в гору
Лошадка, везущая хворосту воз.

```
PS > git commit -am "Закончил стих"
[wip eec7966] Закончил стих
1 file changed, 3 insertions(+), 1 deletion(-)
```



Папка

ch1.txt(4) ch2.txt(2
→ 3)

Индекс

ch1.txt(4) ch2.txt(2
→ 3)

Репозиторий

ch1.txt(4) ch2.txt(2
→ 3)

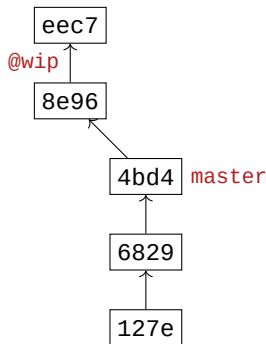
Удаление веток (DANGER!!)

```
git branch -D имя_ветки —
```

принудительное удаление ветки. Команда принудительно удаляет указанную ветку без проверки того, слиты ли её изменения в текущую или другую ветку.

```
git reflog —
```

журнал ссылок (Reference Log). Команда показывает историю перемещений указателей (HEAD, веток) в локальном репозитории. Это локальный журнал всех действий, которые меняли положение веток.



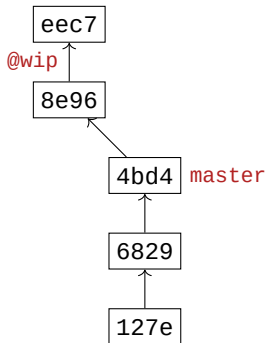
Папка	Индекс	Репозиторий
ch1.txt(4) ch2.txt(3)	ch1.txt(4) ch2.txt(3)	ch1.txt(4) ch2.txt(3)

Удаление веток (DANGER!!)

2. Удаляем ветку

```
PS > git branch -D wip  
error: cannot delete branch 'wip' used by worktree
```

```
PS > git checkout master  
PS > git branch -D wip  
Deleted branch wip (was eec7966).
```



Папка
ch1.txt(4) ch2.txt(3)

Индекс
ch1.txt(4) ch2.txt(3)

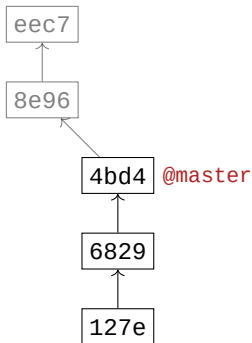
Репозиторий
ch1.txt(4) ch2.txt(3)

Удаление веток (DANGER!!)

2. Удаляем ветку

```
PS > git branch -D wip  
error: cannot delete branch 'wip' used by workspace
```

```
PS > git checkout master  
PS > git branch -D wip  
Deleted branch wip (was eec7966).
```



Папка
ch1.txt(4) ch2.txt(3)

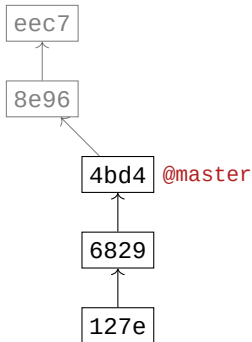
Индекс
ch1.txt(4) ch2.txt(3)

Репозиторий
ch1.txt(4) ch2.txt(3)

Удаление веток (DANGER!!)

3. Пытаемся починить

```
PS > git reflog
4bd412d (HEAD -> master) HEAD@0: checkout: moving from
eec7966 HEAD@1: commit: Закончил стих
8e965b9 HEAD@2: checkout: moving from master to wip
4bd412d (HEAD -> master) HEAD@3: checkout: moving from
8e965b9 HEAD@4: checkout: moving from master to wip
4bd412d (HEAD -> master) HEAD@5: checkout: moving from
```



Папка

ch1.txt(4) ch2.txt(3)

Индекс

ch1.txt(4) ch2.txt(3)

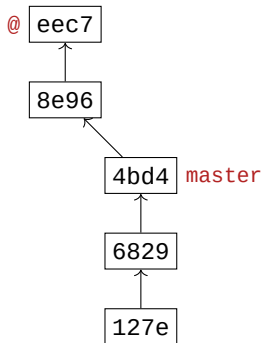
Репозиторий

ch1.txt(4) ch2.txt(3)

Удаление веток (DANGER!!)

```
PS > git checkout eec7
Note: switching to 'eec7'.
```

```
You are in 'detached HEAD' state...и еще много текста
...
HEAD is now at eec7966 Закончил стих
```



Detached HEAD —

это состояние в Git, когда указатель HEAD ссылается не на ветку (не на имя ветки, например main или develop), а непосредственно на конкретный коммит

Папка
ch1.txt(4) ch2.txt(3)

Индекс
ch1.txt(4) ch2.txt(3)

Репозиторий
ch1.txt(4) ch2.txt(3)

Приложения

Просмотр лога

```
> git log
commit 4bd412d5bfbeeb8546f9e383ee6319802d7c2bee3 (HEAD -> master)
Author: gavreg <a.gavrilov@stankin.ru>
Date: Thu Mar 19 00:12:14 2026 +0300
```

```
КОММИТ номер три
commit 68297fe79d35daed901bcae1f8819b03ec7f5dbd
Author: gavreg <a.gavrilov@stankin.ru>
Date: Wed Mar 18 23:24:23 2026 +0300
```

Второй коммит

```
commit 127ec2cc61950b26931bf323c902b1d5cc9b979d
Author: gavreg <a.gavrilov@stankin.ru>
Date: Wed Mar 18 19:32:58 2026 +0300
```

Мой первый коммит

К слайду

git show 127e

```
> git show 127e
commit 127ec2cc61950b26931bf323c902b1d5cc9b979d
Author: gavreg <a.gavrilov@stankin.ru>
Date:   Wed Mar 18 19:32:58 2026 +0300
```

Мой первый коммит

```
diff --git a/ch1.txt b/ch1.txt
new file mode 100644
index 0000000..e0b0dee
--- /dev/null
+++ b/ch1.txt
@@ -0,0 +1 @@
+Вечерело, на улице было холодно.
\ No newline at end of file
```

К слайду

git show 6829

```
> git show 6829
```

```
...
```

Второй коммит

```
diff --git a/ch1.txt b/ch1.txt
```

```
index e0b0dee..ccceeb9 100644
```

```
--- a/ch1.txt
```

```
+++ b/ch1.txt
```

```
@@ -1,1 @@
```

```
-Вечерело, на улице было холодно.
```

```
\ No newline at end of file
```

```
+Вечерело, на улице было жарко.
```

```
\ No newline at end of file
```

```
diff --git a/ch2.txt b/ch2.txt
```

```
new file mode 100644
```

```
index 0000000..3f3ac29
```

```
--- /dev/null
```

```
+++ b/ch2.txt
```

```
@@ -0,0 +1 @@
```

```
+Однажды, в студеную зимнюю пору я из лесу вышел.
```

К слайду

git show 4bd4

```
> git show 4bd4
...
Коммит номер три

diff --git a/ch1.txt b/ch1.txt
index ccceeb9..14fdde3 100644
--- a/ch1.txt
+++ b/ch1.txt
@@ -1 +1 @@
-Вечерело, на улице было жарко.
\ No newline at end of file
+Вечерело, на улице шёл дождь
\ No newline at end of file
diff --git a/ch2.txt b/ch2.txt
index 3f3ac29..2c56d7c 100644
--- a/ch2.txt
+++ b/ch2.txt
@@ -1 +1,2 @@
-Однажды, в студеную зимнюю пору я из лесу вышел.
\ No newline at end of file
+Однажды, в студеную зимнюю пору
+Я из лесу вышел; был сильный мороз.
\ No newline at end of file
```

К слайду